

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Ingeniería de Software

Cliente Node.js para servicios web de Spring Boot

Guía paso a paso: consumir una API REST con Express, EJS, Axios y fetch, con autenticación JWT

Entorno de desarrollo: IntelliJ IDEA. El servidor Spring Boot se asume ya construido y en ejecución.

Proyecto: aplicación web cliente con inicio de sesión JWT, listado, alta, edición y baja consumiendo la API

Asignatura: Aplicaciones Web (5.º semestre)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Versiones de referencia (Node.js 22 LTS, Express 5, IntelliJ IDEA 2025) en la sección 3.

Contenido

1	Introducción	3
1.1	Qué significa «consumir un servicio web»	3
1.2	El estilo REST y por qué se usa	3
1.3	El formato de los datos: JSON	4
1.4	Lo que construirás	4
2	Arquitectura y supuestos del servidor	5
2.1	Dirección, puerto y endpoints	5
2.2	El patrón: un cliente web que actúa de intermediario	5
3	Preparación del entorno en IntelliJ IDEA	6
3.1	Instalar Node.js y habilitar el soporte en IntelliJ	6
3.2	Versiones de referencia (verificadas a julio de 2026)	6
4	Paso 1: crear el proyecto Node.js	7
4.1	Inicializar el proyecto	7
4.2	El archivo package.json	8
4.3	La estructura tras el paso 1	8
5	Paso 2: instalar las dependencias	8
6	Paso 3: configuración y variables de entorno	10
7	Paso 4: la capa de servicios (Axios y fetch comparados)	11
7.1	Opción A: un cliente Axios con URL base	11
7.2	Opción B: el mismo servicio con fetch nativo	12
8	Paso 5: autenticación con JWT	13
8.1	Obtener el token: el inicio de sesión	13
8.2	Enviar el token: la cabecera Authorization	14
9	Paso 6: el servidor Express y la sesión	15
9.1	Un middleware para proteger rutas	16
10	Paso 7: las rutas de inicio de sesión	17

11 Paso 8: el CRUD consumiendo la API	18
11.1 El servicio de estudiantes	19
11.2 Las rutas del CRUD	19
12 Paso 9: las vistas con EJS	21
12.1 La vista de login	21
12.2 La vista del listado	22
12.3 La vista del formulario	23
12.4 Una hoja de estilos para la carpeta pública	24
13 Paso 10: manejo de errores y CORS	25
13.1 Un middleware de manejo de errores	25
13.2 Qué es CORS y por qué aquí no molesta	26
14 Paso 11: ejecutar el cliente en IntelliJ IDEA	26
14.1 Arrancar desde la terminal	26
14.2 Una configuración de ejecución para depurar	27
15 El proyecto completo	27
16 Glosario	28
17 Recursos	29

1 Introducción

Esta guía te enseña a construir, paso a paso, una aplicación cliente en Node.js que consume los servicios web de un servidor hecho en Spring Boot. Partimos de un supuesto explícito: el servidor ya existe, expone una API REST y está en ejecución; nuestro trabajo es escribir el cliente (client, «el que pide») que consume esa API. Construiremos una aplicación web completa con inicio de sesión mediante JWT, y con listado, alta, edición y baja de registros, todo alimentado por las llamadas al servidor.

El método es práctico y acumulativo: cada sección plantea primero el problema concreto que resuelve, explica la técnica y muestra un ejemplo pequeño pero completo. Al final reunimos todas las piezas en una aplicación que funciona de extremo a extremo. Todo el desarrollo se hace en el IDE (Integrated Development Environment, «entorno de desarrollo integrado») IntelliJ IDEA.

Convención sobre los términos

Cada término nuevo propio de las aplicaciones web se explica la primera vez que aparece. Cuando el término está en inglés, su significado en español se indica entre paréntesis, así: endpoint (punto de acceso). Al final de la guía encontrarás un glosario para consulta rápida.

1.1 Qué significa «consumir un servicio web»

Un servicio web (web service) es un programa que otro programa puede usar a través de la red, enviando peticiones y recibiendo respuestas en un formato acordado. Cuando decimos que nuestro cliente «consume» el servicio, queremos decir que le envía peticiones HTTP y procesa las respuestas que devuelve. El servidor Spring Boot ofrece los datos y las operaciones; el cliente Node.js los pide y los presenta al usuario en páginas web.

En esta arquitectura hay una separación clara de responsabilidades: el servidor (server, «el que sirve») contiene la lógica de negocio y la base de datos; el cliente contiene la interfaz de usuario y orquesta las llamadas. Ambos se comunican mediante una API REST, que es el contrato que define qué operaciones existen y cómo invocarlas.

1.2 El estilo REST y por qué se usa

REST (Representational State Transfer, «transferencia de estado representacional») es un estilo de arquitectura para diseñar servicios web sobre HTTP, formalizado por Roy Fielding en su tesis doctoral [1]. En una API REST cada recurso (resource, «recurso»: por ejemplo, un estudiante) se identifica con una URL (Uniform Resource Locator, «localizador uniforme de recursos») y se manipula con los verbos (verbs, «métodos») de HTTP. Es el estilo dominante para comunicar un cliente con un servidor.

La semántica de esos verbos y de los códigos de estado está definida en el estándar HTTP vigente [2]. Los cuatro verbos que usaremos son GET (obtener), POST (crear), PUT (actualizar) y DELETE (eliminar). El servidor responde con un código de estado (status code) que indica el resultado: por ejemplo, 200 (OK, «correcto»), 201 (Created, «creado»), 401 (Unauthorized, «no autorizado») o 404 (Not Found, «no encontrado»).

Verbo	Significado	Ejemplo sobre el recurso <code>estudiantes</code>
GET	Obtener datos	GET <code>/api/estudiantes</code> lista todos.
POST	Crear un recurso	POST <code>/api/estudiantes</code> crea uno nuevo.
PUT	Actualizar un recurso	PUT <code>/api/estudiantes/5</code> edita el de id 5.
DELETE	Eliminar un recurso	DELETE <code>/api/estudiantes/5</code> borra el de id 5.

1.3 El formato de los datos: JSON

Los datos viajan entre cliente y servidor en formato JSON (JavaScript Object Notation, «notación de objetos de JavaScript»), un formato de texto ligero para intercambiar datos estructurados, definido en el estándar [3]. Un estudiante se representa así en JSON: un objeto con pares de nombre y valor. Node.js convierte automáticamente entre JSON (texto) y objetos de JavaScript, lo que hace muy cómodo trabajar con la API.

Un recurso «estudiante» en formato JSON

```

1 {
2   "id": 5,
3   "nombre": "Ana Torres",
4   "correo": "ana.torres@uteq.edu.ec",
5   "carrera": "Ingenieria de Software"
6 }
```

Convertir un objeto de JavaScript a texto JSON se llama serializar (`serialize`), y el paso inverso, deserializar (`deserialize`). En el cliente casi nunca lo harás a mano: la librería HTTP y Express lo hacen por ti, como veremos.

1.4 Lo que construirás

Al terminar tendrás una aplicación web cliente, servida por Express, que consume la API REST de Spring Boot. En concreto sabrás: iniciar sesión contra el servidor y obtener un token JWT; enviar ese token en cada petición protegida; y realizar el CRUD (Create, Read, Update, Delete: «crear, leer, actualizar, eliminar») completo de estudiantes consumiendo los endpoints del servidor, mostrando los resultados en vistas HTML.

2 Arquitectura y supuestos del servidor

Problema a resolver

Antes de escribir una sola línea del cliente necesitamos saber exactamente qué expone el servidor: en qué dirección y puerto responde, qué endpoints ofrece y cómo se autentica. ¿Cuáles son esos supuestos?

Como el servidor Spring Boot ya está hecho, fijamos aquí los supuestos que usaremos en toda la guía. Un endpoint (punto de acceso o extremo) es cada dirección concreta de la API que atiende un tipo de petición. La dirección base (base URL, «URL base») del servidor es la parte común a todos los endpoints; el cliente la usará como prefijo.

2.1 Dirección, puerto y endpoints

Asumimos que el servidor Spring Boot escucha en el puerto 8080 de la misma máquina, por lo que su URL base es `http://localhost:8080`. El término `localhost` («equipo local») designa tu propia computadora; el número que sigue a los dos puntos es el puerto (port), un canal numerado por el que el servidor atiende. Los endpoints de la API cuelgan del prefijo `/api`.

Endpoint	Verbo	Qué hace
<code>/api/auth/login</code>	POST	Recibe usuario y clave; devuelve un token JWT.
<code>/api/estudiantes</code>	GET	Devuelve la lista de estudiantes.
<code>/api/estudiantes/{id}</code>	GET	Devuelve un estudiante por su id.
<code>/api/estudiantes</code>	POST	Crea un estudiante con los datos del cuerpo.
<code>/api/estudiantes/{id}</code>	PUT	Actualiza el estudiante indicado.
<code>/api/estudiantes/{id}</code>	DELETE	Elimina el estudiante indicado.

El cuerpo (body, «cuerpo» o payload, «carga útil») de una petición POST o PUT es el JSON con los datos que enviamos; el cuerpo de la respuesta es el JSON que devuelve el servidor. Nuestro cliente Node.js correrá en el puerto 3000, es decir en `http://localhost:3000`, para no chocar con el servidor.

2.2 El patrón: un cliente web que actúa de intermediario

Nuestro cliente no es una simple pantalla: es un servidor web Express que, a su vez, es cliente de la API de Spring Boot. Este patrón se conoce como BFF (Backend for Frontend, «backend para el frontend»): un backend (parte del servidor) pequeño y propio que sirve las páginas al navegador y concentra las llamadas a la API. El navegador habla con Express; Express habla con Spring Boot.

Flujo de una petición de extremo a extremo

```

1 Navegador  --(HTTP)-->  Cliente Node.js/Express  --(HTTP + JWT)-->  Servidor Spring Boot
2   (vistas EJS)           localhost:3000                localhost:8080/api

```

Por qué un intermediario y no llamar a la API desde el navegador

Colocar Express en medio nos permite guardar el token JWT en la sesión del servidor (fuera del alcance de scripts del navegador), evitar problemas de CORS y no exponer detalles de la API. Es un diseño más seguro y ordenado, y es el que la industria llama BFF.

3 Preparación del entorno en IntelliJ IDEA

Problema a resolver

Para escribir y ejecutar el cliente necesitamos el motor de Node.js, un gestor de paquetes y el IDE configurado para reconocerlos. ¿Cómo preparamos ese entorno en IntelliJ IDEA?

Node.js es el runtime (entorno de ejecución) que ejecuta JavaScript fuera del navegador; con él, JavaScript sirve para escribir servidores. Junto a Node.js se instala npm (Node Package Manager, «gestor de paquetes de Node»), la herramienta que descarga e instala las librerías (llamadas paquetes o dependencias) de las que depende el proyecto.

3.1 Instalar Node.js y habilitar el soporte en IntelliJ

Instala una versión LTS (Long-Term Support, «soporte a largo plazo») de Node.js —la 22— desde nodejs.org; el instalador incluye npm. En IntelliJ IDEA, activa el plugin (complemento) «Node.js» desde *File* → *Settings* → *Plugins*, y comprueba que el IDE detecta el intérprete en *Settings* → *Languages & Frameworks* → *Node.js*. Con eso, IntelliJ ofrece autocompletado, ejecución y depuración de código Node.

Comprobación rápida

Abre la terminal integrada de IntelliJ (*View* → *Tool Windows* → *Terminal*) y ejecuta `node -version` y `npm -version`. Si ambos imprimen un número de versión, el entorno está listo.

3.2 Versiones de referencia (verificadas a julio de 2026)

Estas son las versiones vigentes de la tecnología usada en la guía.

Componente	Versión de referencia (jul. 2026)
Node.js (LTS)	22.x (incluye <code>fetch</code> nativo, disponible desde Node 18).

Componente	Versión de referencia (jul. 2026)
<code>npm</code>	10.x (viene con Node.js).
<code>Express</code>	5.x (framework web del cliente).
<code>EJS</code>	3.x (motor de plantillas para las vistas).
<code>Axios</code>	1.x (cliente HTTP con interceptores).
<code>IntelliJ IDEA</code>	2025.x (Community o Ultimate, con el plugin Node.js).

4 Paso 1: crear el proyecto Node.js

Problema a resolver

Necesitamos un proyecto Node.js con su archivo de configuración y una estructura de carpetas ordenada. ¿Cómo lo creamos desde IntelliJ IDEA?

Todo proyecto Node.js se describe en un archivo `package.json`, que declara su nombre, sus scripts de arranque y sus dependencias. Lo genera el comando `npm init`. Crearemos el proyecto y, dentro de él, las carpetas que separan responsabilidades.

4.1 Inicializar el proyecto

En IntelliJ, crea un proyecto vacío en una carpeta llamada `cliente-estudiantes` y abre la terminal integrada en esa carpeta. Ejecuta `npm init -y` para generar un `package.json` inicial (la opción `-y` acepta los valores por defecto). Después crea la estructura de carpetas del proyecto.

Crear en: Terminal integrada de IntelliJ (en la raíz del proyecto)

Inicializar el proyecto y crear carpetas

```
1 npm init -y
2 mkdir src src/rutas src/servicios src/vistas src/publico
```

La carpeta `src` contendrá el código; `rutas` las rutas de Express; `servicios` el código que llama a la API; `vistas` las plantillas EJS; y `publico` los archivos estáticos (CSS, imágenes). Esta separación mantiene el proyecto legible a medida que crece.

Crear carpetas sin la terminal

Si prefieres el ratón, cada carpeta se crea con clic secundario (botón derecho) sobre la carpeta contenedora en el panel *Project* de IntelliJ → *New* → *Directory*. Por ejemplo, `src` se crea sobre la raíz `cliente-estudiantes`, y `rutas` se crea sobre `src`.

4.2 El archivo package.json

Ajustamos el `package.json` para indicar el punto de entrada y un script de arranque. La clave `"type": "module"` habilita la sintaxis moderna de módulos de JavaScript (`import/export`), más clara que la antigua `require` [4]. El script `start` permite arrancar la aplicación con `npm start`.

Dónde editarlo: abre el archivo existente (lo generó `npm init -y`); ábrelo y reemplaza su contenido por este.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/package.json`

package.json

```

1 {
2   "name": "cliente-estudiantes",
3   "version": "1.0.0",
4   "type": "module",
5   "main": "src/app.js",
6   "scripts": {
7     "start": "node src/app.js",
8     "dev": "node --watch src/app.js"
9   }
10 }
```

El script `dev` usa `-watch` (observar), que reinicia el proceso automáticamente al guardar cambios: muy cómodo durante el desarrollo.

4.3 La estructura tras el paso 1

Con el proyecto inicializado y las carpetas creadas, esta es la estructura de partida. A partir de aquí, cada paso añadirá archivos y volveremos a mostrar el árbol marcando lo nuevo.

Estructura tras el paso 1

```

1 cliente-estudiantes/
2 |- package.json      # generado y ajustado en el paso 1
3 |- src/
4   |- rutas/          # (vacía por ahora)
5   |- servicios/      # (vacía por ahora)
6   |- vistas/         # (vacía por ahora)
7   |- publico/        # (vacía por ahora)
```

5 Paso 2: instalar las dependencias

Problema a resolver

El cliente necesita un framework web (Express), un motor de plantillas (EJS), un cliente HTTP (Axios), manejo de sesión y variables de entorno. ¿Qué dependencias instalamos y para qué sirve cada una?

Una dependencia (dependency) es una librería externa que nuestro proyecto usa. Se instala con **npm install**, que la descarga y la registra en **package.json**. Instalamos de una vez las que necesita el cliente. No instalamos ningún cliente HTTP adicional para **fetch**, porque Node.js 22 ya lo incluye de forma nativa [5].

Crear en: Terminal integrada de IntelliJ

Instalar dependencias

```
1 npm install express ejs axios express-session dotenv
```

Paquete	Para qué sirve
express	Framework web: define rutas y sirve las páginas del cliente.
ejs	Motor de plantillas: genera HTML con datos dinámicos.
axios	Cliente HTTP con interceptores, para llamar a la API.
express-session	Guarda datos por usuario (el token JWT) entre peticiones.
dotenv	Carga variables de entorno desde un archivo .env .

Un framework (marco de trabajo) es una librería que impone una estructura y resuelve las tareas comunes para que te centres en tu lógica. Express es el framework web más usado en Node.js. Un motor de plantillas (template engine) es la herramienta que combina una plantilla HTML con datos para producir la página final; usaremos EJS (Embedded JavaScript, «JavaScript incrustado»).

Al instalar, npm crea automáticamente dos elementos en la raíz: la carpeta **node_modules** (donde descarga las librerías) y el archivo **package-lock.json** (que fija las versiones exactas). No los creas tú ni los subes al repositorio.

Estructura tras el paso 2

```
1 cliente-estudiantes/
2 |- package.json          # ahora incluye las dependencias instaladas
3 |- package-lock.json     # <-- nuevo (lo genera npm; no se versiona)
4 |- node_modules/        # <-- nuevo (librerías; no se versiona)
5 |- src/
6   |- rutas/
7   |- servicios/
8   |- vistas/
9   |- publico/
```

6 Paso 3: configuración y variables de entorno

Problema a resolver

La URL del servidor y otros valores sensibles no deben estar «quemados» en el código. ¿Cómo los configuramos de forma limpia y segura?

Una variable de entorno (environment variable) es un valor de configuración que vive fuera del código, en el entorno donde se ejecuta la aplicación. Las guardamos en un archivo `.env` y las leemos con el paquete `dotenv`. Así, cambiar la URL del servidor no obliga a tocar el código, y los secretos no acaban en el repositorio.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `cliente-estudiantes` (la raíz) → *New* → *File* → *nombre .env*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/.env`

.env

```
1 API_BASE_URL=http://localhost:8080/api
2 SESSION_SECRET=cambia-esta-clave-por-una-larga-y-aleatoria
3 PORT=3000
```

No subas el archivo `.env` al repositorio

El `.env` contiene secretos. Añade una línea con `.env` a tu archivo `.gitignore` para que no se suba al repositorio. En su lugar, se suele versionar un `.env.example` con las claves pero sin los valores reales.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `cliente-estudiantes` (la raíz) → *New* → *File* → *nombre .gitignore*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/.gitignore`

.gitignore

```
1 node_modules/
2 .env
```

Estructura tras el paso 3

```

1 cliente-estudiantes/
2 |- package.json
3 |- package-lock.json
4 |- .env                # <-- nuevo (configuracion y secretos)
5 |- .gitignore          # <-- nuevo (excluye node_modules y .env)
6 |- node_modules/
7 |- src/
8   |- rutas/
9   |- servicios/
10  |- vistas/
11  |- publico/

```

7 Paso 4: la capa de servicios (Axios y fetch comparados)

Problema a resolver

Todas las llamadas a la API deben concentrarse en un solo lugar, con la URL base y las cabeceras comunes ya configuradas. ¿Cómo construimos esa capa, y qué diferencia hay entre usar Axios y el `fetch` nativo?

Concentraremos el acceso a la API en una «capa de servicios»: módulos cuya única tarea es llamar al servidor y devolver datos. Compararemos dos formas de hacer las peticiones HTTP: Axios (una librería) y `fetch` (la función nativa de Node.js). Ambas devuelven una promesa (promise, «promesa»: un objeto que representa un resultado que llegará más tarde), por lo que se usan con `async/await` («asíncrono»/«esperar»), la sintaxis que permite escribir código asíncrono de forma secuencial y legible.

7.1 Opción A: un cliente Axios con URL base

Axios permite crear una «instancia» preconfigurada con la URL base y un tiempo máximo de espera (timeout). Así, en el resto del código solo indicamos la parte final del endpoint. Axios serializa y deserializa el JSON automáticamente: lo que envías en `data` se convierte a JSON, y la respuesta llega ya convertida en `response.data`.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/servicios` → *New* → *JavaScript File* → nombre *apiCliente*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/servicios/apiCliente.js`

src/servicios/apiCliente.js (instancia de Axios)

```

1 import axios from "axios";
2 import "dotenv/config";
3
4 // Instancia con la URL base tomada de la variable de entorno
5 const api = axios.create({
6   baseURL: process.env.API_BASE_URL,    // http://localhost:8080/api
7   timeout: 8000,                        // 8 s de espera máxima
8   headers: { "Content-Type": "application/json" }
9 });
10
11 export default api;

```

La cabecera (header) `Content-Type: application/json` le dice al servidor que el cuerpo que enviamos está en formato JSON. Una cabecera es un par nombre–valor que acompaña a la petición o a la respuesta con información de control (tipo de contenido, autenticación, etc.).

7.2 Opción B: el mismo servicio con fetch nativo

La función `fetch` hace lo mismo sin instalar nada, pero es de más bajo nivel: no tiene URL base (hay que componer la URL completa), no lanza error automáticamente ante códigos como 404 o 500 (debes revisar `response.ok`) y no convierte el JSON solo (debes llamar a `response.json()`). El siguiente ayudante encapsula esas diferencias.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/servicios` → *New* → *JavaScript File* → nombre `apiFetch`.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/servicios/apiFetch.js`

src/servicios/apiFetch.js (envoltorio sobre fetch)

```

1 import "dotenv/config";
2 const BASE = process.env.API_BASE_URL;
3
4 // Envoltorio: compone la URL, fija cabeceras y valida la respuesta
5 export async function pedir(ruta, opciones = {}) {
6   const respuesta = await fetch(BASE + ruta, {
7     headers: { "Content-Type": "application/json", ...(opciones.headers || {}) },
8     ...opciones
9   });
10   if (!respuesta.ok) { // fetch NO lanza error solo
11     throw new Error("HTTP " + respuesta.status);
12   }
13   if (respuesta.status === 204) return null; // 204 = sin contenido
14   return await respuesta.json(); // deserializa el JSON
15 }

```

Axios o fetch: cuál elegir

Para este proyecto usaremos Axios por comodidad: su URL base, sus interceptores y el manejo automático de errores y de JSON ahorran código repetitivo. `fetch` es perfecto cuando quieres cero dependencias. Sabiendo que ambos devuelven promesas y se usan con `await`, cambiar de uno a otro es casi mecánico.

Aspecto	Axios	fetch nativo
Instalación	Dependencia externa	Incluido en Node 18+
URL base	Sí, en la instancia	No, URL completa
JSON	Automático	Manual (<code>.json()</code>)
Error por 4xx/5xx	Automático	Manual (<code>response.ok</code>)
Interceptores	Sí	No (a mano)

Estructura tras el paso 4

```

1 cliente-estudiantes/
2 |- package.json package-lock.json .env .gitignore
3 |- node_modules/           # (omitido en adelante)
4 |- src/
5   |- rutas/
6   |- servicios/
7     |- apiCliente.js       # <-- nuevo (instancia de Axios)
8     |- apiFetch.js         # <-- nuevo (envoltorio sobre fetch)
9   |- vistas/
10  |- publico/

```

8 Paso 5: autenticación con JWT

Problema a resolver

Los endpoints de estudiantes están protegidos: el servidor exige un token que demuestre que el usuario inició sesión. ¿Cómo obtenemos ese token y cómo lo enviamos en cada petición?

La API usa JWT (JSON Web Token, «token web JSON»), un formato compacto y firmado para transmitir de forma segura afirmaciones sobre un usuario, definido en el estándar [6]. Un token (ficha o credencial) es una cadena que el servidor emite al iniciar sesión y que el cliente presenta después para probar su identidad. El flujo tiene dos momentos: primero se obtiene el token; luego se envía en cada petición protegida.

8.1 Obtener el token: el inicio de sesión

El servicio de autenticación envía las credenciales al endpoint `/auth/login` con POST y recibe el token en la respuesta. Aquí lo hacemos con la instancia de Axios de la sección anterior. Devolvemos el token para que quien llame lo guarde.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/servicios` → *New* → *JavaScript File* → nombre *authServicio*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/servicios/authServicio.js`

`src/servicios/authServicio.js`

```
1 import api from "./apiCliente.js";
2
3 // Envía usuario y clave; devuelve el token JWT que emite el servidor
4 export async function iniciarSesion(usuario, clave) {
5   const respuesta = await api.post("/auth/login", { usuario, clave });
6   return respuesta.data.token; // el servidor responde { "token": "..."}
7 }
```

8.2 Enviar el token: la cabecera Authorization

Una vez obtenido, el token se envía en cada petición protegida dentro de la cabecera **Authorization** (autorización), con el esquema Bearer (portador): el valor es la palabra **Bearer** seguida de un espacio y el token. Para no repetir esto en cada llamada, usamos un interceptor (interceptor): una función de Axios que se ejecuta antes de cada petición y le añade la cabecera automáticamente si hay token.

Dónde editarlo: abre el archivo existente `apiCliente.js` y añade este bloque al final (no crees un archivo nuevo).

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/servicios/apiCliente.js`

Interceptor que añade el token a cada petición

```
1 // El token se guarda por petición en api.defaults; ver el paso 6
2 api.interceptors.request.use((config) => {
3   if (api.tokenActual) {
4     config.headers.Authorization = "Bearer " + api.tokenActual;
5   }
6   return config;
7 });
```

Buenas prácticas con el token (RFC 8725)

El token debe viajar siempre por HTTPS, tener una caducidad corta y validarse en el servidor; no debe guardarse en el almacenamiento del navegador accesible por scripts [7, 8]. Por eso en esta guía lo guardamos en la sesión del servidor Express (paso 6), no en el navegador.

Estructura tras el paso 5

```
1 cliente-estudiantes/  
2 |- src/  
3   |- rutas/  
4   |- servicios/  
5     |- apiCliente.js      # se le agrego el interceptor del token  
6     |- apiFetch.js  
7     |- authServicio.js    # <-- nuevo (iniciarSesion)  
8   |- vistas/  
9   |- publico/
```

9 Paso 6: el servidor Express y la sesión

Problema a resolver

Necesitamos un servidor web que sirva las páginas, recuerde el token de cada usuario entre peticiones y enrute las acciones. ¿Cómo lo montamos con Express?

Express organiza la aplicación en middlewares y rutas. Un middleware (software intermedio) es una función que procesa la petición antes de que llegue a su destino: aquí usaremos middlewares para leer formularios, servir archivos estáticos y manejar la sesión. Una ruta (route) asocia un verbo y una URL con el código que responde. La sesión (session) es un almacén por usuario en el servidor donde guardaremos el token JWT.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src` → *New* → *JavaScript File* → *nombre app*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/app.js`

src/app.js (configuración del servidor)

```

1 import express from "express";
2 import session from "express-session";
3 import "dotenv/config";
4 import rutasAuth from "../rutas/auth.js";
5 import rutasEstudiantes from "../rutas/estudiantes.js";
6
7 const app = express();
8
9 // Middlewares base
10 app.set("view engine", "ejs"); // motor de plantillas EJS
11 app.set("views", "src/vistas");
12 app.use(express.urlencoded({ extended: true })); // lee formularios HTML
13 app.use(express.static("src/publico")); // sirve CSS e imagenes
14 app.use(session({
15   secret: process.env.SESSION_SECRET,
16   resave: false,
17   saveUninitialized: false
18 }));
19
20 // Rutas
21 app.use("/", rutasAuth);
22 app.use("/estudiantes", rutasEstudiantes);
23
24 const PORT = process.env.PORT || 3000;
25 app.listen(PORT, () => console.log("Cliente en http://localhost:" + PORT));

```

El middleware `express.urlencoded` deserializa los datos de los formularios HTML y los deja en `req.body`. El middleware `session` crea el almacén de sesión. La llamada `app.listen` arranca el servidor en el puerto configurado.

9.1 Un middleware para proteger rutas

Igual que el servidor exige token, nuestro cliente debe impedir el acceso a las páginas privadas si el usuario no ha iniciado sesión. Un middleware de guarda revisa si hay token en la sesión: si lo hay, deja pasar; si no, redirige al inicio de sesión.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/rutas` → *New* → *JavaScript File* → *nombre guardia*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/rutas/guardia.js`

src/rutas/guardia.js (control de acceso)

```

1 // Deja pasar solo si hay token en la sesion; si no, va al login
2 export function requiereSesion(req, res, next) {
3   if (req.session && req.session.token) {
4     return next(); // continua hacia la ruta
5   }
6   return res.redirect("/login");
7 }

```

La función **next** (siguiente) es el mecanismo de Express para pasar el control al siguiente middleware o a la ruta. Llamarla significa «todo bien, continúa»; no llamarla y responder (como en el **redirect**) corta la cadena.

Estructura tras el paso 6

```
1 cliente-estudiantes/  
2 |- src/  
3   |- app.js                # <-- nuevo (servidor Express)  
4   |- rutas/  
5     |- guardia.js         # <-- nuevo (requiereSesion)  
6   |- servicios/  
7     |- apiCliente.js  apiFetch.js  authService.js  
8   |- vistas/  
9   |- publico/
```

10 Paso 7: las rutas de inicio de sesión

Problema a resolver

Necesitamos una página de inicio de sesión, procesar sus datos contra la API, guardar el token en la sesión y ofrecer el cierre de sesión. ¿Cómo se escriben esas rutas?

Estas rutas conectan el formulario de login con el servicio de autenticación del paso 5. Al enviar el formulario, la ruta llama a **iniciarSesion**, guarda el token devuelto en **req.session** y redirige al listado. El cierre de sesión destruye la sesión.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre **src/rutas** → *New* → *JavaScript File* → nombre **auth**.

Ruta (desde la raíz del proyecto): **cliente-estudiantes/src/rutas/auth.js**

src/rutas/auth.js

```

1 import { Router } from "express";
2 import { iniciarSesion } from "../servicios/authServicio.js";
3
4 const router = Router();
5
6 // Muestra el formulario de login
7 router.get("/login", (req, res) => res.render("login", { error: null }));
8
9 // Procesa el login: pide el token a la API y lo guarda en la sesion
10 router.post("/login", async (req, res) => {
11   try {
12     const { usuario, clave } = req.body;
13     const token = await iniciarSesion(usuario, clave);
14     req.session.token = token; // se guarda en el servidor
15     res.redirect("/estudiantes");
16   } catch (e) {
17     res.status(401).render("login", { error: "Credenciales invalidas" });
18   }
19 });
20
21 // Cierra la sesion
22 router.get("/logout", (req, res) => {
23   req.session.destroy(() => res.redirect("/login"));
24 });
25
26 export default router;

```

El objeto Router (enrutador) agrupa rutas relacionadas en un módulo. La palabra **await** espera a que la promesa de **iniciarSesion** se resuelva antes de continuar; si el servidor rechaza las credenciales, Axios lanza un error que el **try/catch** captura para mostrar el mensaje.

Estructura tras el paso 7

```

1 cliente-estudiantes/
2 |- src/
3   |- app.js
4   |- rutas/
5     |- guardia.js
6     |- auth.js           # <-- nuevo (login, logout)
7   |- servicios/
8     |- apiCliente.js  apiFetch.js  authServicio.js
9   |- vistas/
10  |- publico/

```

11 Paso 8: el CRUD consumiendo la API

Problema a resolver

El corazón del cliente es listar, crear, editar y eliminar estudiantes llamando a los endpoints del servidor, enviando el token en cada petición. ¿Cómo se implementa ese CRUD?

Resolvemos el CRUD en dos piezas: un servicio que hace las llamadas a la API (reutilizando la instancia de Axios y su interceptor) y unas rutas que reciben las peticiones del navegador, pasan el token de la sesión al servicio y renderizan las vistas. Empezamos por el servicio.

11.1 El servicio de estudiantes

Cada método corresponde a un endpoint y a un verbo. Recibimos el token como parámetro y lo colocamos en `api.tokenActual` para que el interceptor lo añada. Axios convierte la respuesta en objetos listos para usar.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/servicios` → *New* → *JavaScript File* → nombre *estudianteServicio*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/servicios/estudianteServicio.js`

`src/servicios/estudianteServicio.js`

```

1 import api from "../apiCliente.js";
2
3 function conToken(token) { api.tokenActual = token; } // lo usa el interceptor
4
5 export async function listar(token) {
6   conToken(token);
7   const r = await api.get("/estudiantes");
8   return r.data; // arreglo de estudiantes
9 }
10
11 export async function buscar(token, id) {
12   conToken(token);
13   const r = await api.get("/estudiantes/" + id);
14   return r.data;
15 }
16
17 export async function crear(token, estudiante) {
18   conToken(token);
19   await api.post("/estudiantes", estudiante); // 201 Created
20 }
21
22 export async function actualizar(token, id, estudiante) {
23   conToken(token);
24   await api.put("/estudiantes/" + id, estudiante);
25 }
26
27 export async function eliminar(token, id) {
28   conToken(token);
29   await api.delete("/estudiantes/" + id); // 204 No Content
30 }

```

11.2 Las rutas del CRUD

Las rutas usan el middleware `requiereSesion` para exigir inicio de sesión, leen el token de la sesión y delegan en el servicio. Tras crear, actualizar o eliminar se redirige al listado: es el patrón

Post-Redirect-Get, que evita que recargar la página repita la operación.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/rutas` → *New* → *JavaScript File* → *nombre estudiantes*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/rutas/estudiantes.js`

src/rutas/estudiantes.js — parte 1/2

```

1 import { Router } from "express";
2 import { requiereSesion } from "../guardia.js";
3 import * as servicio from "../servicios/estudianteServicio.js";
4
5 const router = Router();
6 router.use(requiereSesion);           // todas las rutas exigen sesion
7
8 // Listar
9 router.get("/", async (req, res) => {
10   const estudiantes = await servicio.listar(req.session.token);
11   res.render("lista", { estudiantes });
12 });
13
14 // Mostrar formulario de alta
15 router.get("/nuevo", (req, res) => {
16   res.render("form", { estudiante: null });
17 });
18
19 // Mostrar formulario de edicion
20 router.get("/:id/editar", async (req, res) => {
21   const estudiante = await servicio.buscar(req.session.token, req.params.id);
22   res.render("form", { estudiante });
23 });

```

src/rutas/estudiantes.js — parte 2/2

```

1 // Guardar (crea si no hay id; actualiza si lo hay)
2 router.post("/guardar", async (req, res) => {
3   const { id, nombre, correo, carrera } = req.body;
4   const datos = { nombre, correo, carrera };
5   if (id) {
6     await servicio.actualizar(req.session.token, id, datos);
7   } else {
8     await servicio.crear(req.session.token, datos);
9   }
10   res.redirect("/estudiantes");           // patron PRG
11 });
12
13 // Eliminar
14 router.post("/:id/eliminar", async (req, res) => {
15   await servicio.eliminar(req.session.token, req.params.id);
16   res.redirect("/estudiantes");
17 });
18
19 export default router;

```

El fragmento `:id` en la ruta es un parámetro de ruta (route parameter): un hueco en la URL cuyo valor se lee en `req.params.id`. Así, `/estudiantes/5/editar` entrega 5 como id.

Estructura tras el paso 8

```

1 cliente-estudiantes/
2 |- src/
3   |- app.js
4   |- rutas/
5     |- guardia.js auth.js
6     |- estudiantes.js      # <-- nuevo (CRUD: rutas)
7   |- servicios/
8     |- apiCliente.js apiFetch.js authService.js
9     |- estudianteServicio.js # <-- nuevo (CRUD: llamadas a la API)
10  |- vistas/
11  |- publico/

```

12 Paso 9: las vistas con EJS

Problema a resolver

Necesitamos páginas HTML que muestren la lista, el formulario y el login, insertando los datos que devuelve la API. ¿Cómo las escribimos con EJS?

EJS es un motor de plantillas: archivos HTML con etiquetas especiales para insertar datos y lógica. La etiqueta `<%= ... %>` inserta un valor escapándolo (convirtiendo caracteres peligrosos), lo que previene el ataque XSS (Cross-Site Scripting, «ejecución de scripts entre sitios») [8]. La etiqueta `<% ... %>` ejecuta código (bucles, condicionales) sin imprimir nada.

12.1 La vista de login

Muestra el formulario y, si hubo un error, el mensaje correspondiente. El formulario envía los datos por POST a la ruta `/login`.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/vistas` → *New* → *File* → *nombre login.ejs*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/vistas/login.ejs`

src/vistas/login.ejs

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Iniciar sesion</title></head>
4 <body>
5   <h1>Iniciar sesion</h1>
6   <% if (error) { %>
7     <p style="color:red"><%= error %></p>
8   <% } %>
9   <form method="post" action="/login">
10     <input name="usuario" placeholder="Usuario" required>
11     <input name="clave" type="password" placeholder="Clave" required>
12     <button type="submit">Entrar</button>
13   </form>
14 </body>
15 </html>
```

12.2 La vista del listado

Recorre el arreglo de estudiantes con un bucle y pinta una fila por cada uno, con enlaces para editar y un botón para eliminar. El botón de eliminar va dentro de un formulario POST porque cambia datos en el servidor.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/vistas` → *New* → *File* → *nombre lista.ejs*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/vistas/lista.ejs`

src/vistas/lista.ejs

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
4 <body>
5   <h1>Estudiantes</h1>
6   <a href="/estudiantes/nuevo">Nuevo</a> |
7   <a href="/logout">Cerrar sesion</a>
8   <table border="1">
9     <tr><th>Nombre</th><th>Correo</th><th>Carrera</th><th></th></tr>
10    <% estudiantes.forEach(function (e) { %>
11      <tr>
12        <td><%= e.nombre %></td>
13        <td><%= e.correo %></td>
14        <td><%= e.carrera %></td>
15        <td>
16          <a href="/estudiantes/<%= e.id %>/editar">Editar</a>
17          <form method="post" action="/estudiantes/<%= e.id %>/eliminar"
18            style="display:inline">
19            <button type="submit">Eliminar</button>
20          </form>
21        </td>
22      </tr>
23    <% }); %>
24  </table>
25 </body>
26 </html>

```

12.3 La vista del formulario

Sirve para crear y para editar. Si recibe un estudiante (al editar), rellena los campos y añade un campo oculto con el id; si recibe null, los deja vacíos para un alta. El operador ?. y un valor por defecto evitan errores cuando no hay estudiante.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre src/vistas → *New* → *File* → *nombre form.ejs*.

Ruta (desde la raíz del proyecto): cliente-estudiantes/src/vistas/form.ejs

src/vistas/form.ejs

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiante</title></head>
4 <body>
5   <h1><%= estudiante ? "Editar" : "Nuevo" %> estudiante</h1>
6   <form method="post" action="/estudiantes/guardar">
7     <input type="hidden" name="id" value="<%= estudiante?.id || ' ' %>">
8     <input name="nombre" value="<%= estudiante?.nombre || ' ' %>" required>
9     <input name="correo" value="<%= estudiante?.correo || ' ' %>"
10       type="email" required>
11     <input name="carrera" value="<%= estudiante?.carrera || ' ' %>" required>
12     <button type="submit">Guardar</button>
13   </form>
14   <a href="/estudiantes">Volver</a>
15 </body>
16 </html>

```

12.4 Una hoja de estilos para la carpeta pública

Para que la carpeta `publico` no quede vacía y las páginas tengan un aspecto mínimo, creamos una hoja de estilos. Se sirve gracias al middleware `express.static` del paso 6, y se enlaza desde las vistas con `<link rel="stylesheet" href="/estilos.css">`.

Dónde crearlo en IntelliJ: clic secundario (botón derecho) sobre `src/publico` → *New* → *File* → *nombre estilos.css*.

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/publico/estilos.css`

src/publico/estilos.css

```

1 body { font-family: system-ui, sans-serif; margin: 2rem; color: #1f2933; }
2 table { border-collapse: collapse; }
3 th, td { padding: 6px 10px; }
4 button { cursor: pointer; }

```

Estructura tras el paso 9

```

1 cliente-estudiantes/
2 |- src/
3   |- app.js
4   |- rutas/
5     |- guardia.js  auth.js  estudiantes.js
6   |- servicios/
7     |- apiCliente.js  apiFetch.js  authServiceio.js  estudianteServicio.js
8   |- vistas/
9     |- login.ejs      # <-- nuevo
10    |- lista.ejs      # <-- nuevo
11    |- form.ejs       # <-- nuevo
12  |- publico/
13    |- estilos.css    # <-- nuevo

```

13 Paso 10: manejo de errores y CORS

Problema a resolver

El servidor puede responder con errores (token vencido, datos inválidos, caído) y el navegador puede toparse con restricciones de origen. ¿Cómo tratamos ambos casos?

Un cliente robusto no asume que todo saldrá bien. Trataremos dos frentes: los errores que devuelve la API y el concepto de CORS. Como nuestras llamadas a Spring Boot salen desde el servidor Express (no desde el navegador), CORS casi nunca nos afecta, pero conviene entenderlo.

13.1 Un middleware de manejo de errores

Express reconoce un middleware de error por tener cuatro parámetros. Colócalo al final, después de las rutas, en `app.js`. Distingue el caso del token vencido (el servidor responde 401) para enviar al usuario de vuelta al login.

Dónde editarlo: abre el archivo existente `app.js` e inserta este bloque justo antes de `app.listen(...)` (no crees un archivo nuevo).

Ruta (desde la raíz del proyecto): `cliente-estudiantes/src/app.js`

Middleware de manejo de errores

```

1 // Cuatro parametros: Express lo trata como manejador de errores
2 app.use((err, req, res, next) => {
3   const status = err.response?.status; // codigo que devolvio la API
4   if (status === 401) {
5     return res.redirect("/login"); // token invalido o vencido
6   }
7   console.error(err.message);
8   res.status(500).send("Ocurrio un error al contactar el servidor.");
9 });

```

El operador `?.` (encadenamiento opcional) evita un fallo si `err.response` no existe (por ejemplo, si el servidor está caído y no hubo respuesta). Para que este manejador reciba los errores de las rutas asíncronas en Express 4, envuelve las llamadas en `try/catch` y usa `next(e)`; en Express 5 los errores de funciones `async` se propagan solos.

13.2 Qué es CORS y por qué aquí no molesta

CORS (Cross-Origin Resource Sharing, «intercambio de recursos de origen cruzado») es una norma de seguridad de los navegadores que bloquea las peticiones de una página web hacia un origen (dominio, puerto o protocolo) distinto del suyo, salvo que el servidor lo permita con cabeceras específicas [5]. Afecta al código que corre en el navegador, no al que corre en Node.js.

Ventaja del patrón BFF frente a CORS

Como el navegador solo habla con nuestro Express (mismo origen) y es Express quien llama a Spring Boot desde el servidor, no hay petición «de origen cruzado» en el navegador y CORS no interviene. Si algún día llamaras a la API directamente desde JavaScript del navegador, entonces sí tendrías que habilitar CORS en el servidor Spring Boot.

14 Paso 11: ejecutar el cliente en IntelliJ IDEA

Problema a resolver

Con todo el código escrito, ¿cómo arrancamos y probamos el cliente desde IntelliJ, y cómo lo depuramos si algo falla?

IntelliJ puede ejecutar el proyecto de dos formas: desde la terminal integrada o con una «configuración de ejecución» (run configuration). La segunda permite además depurar (ejecutar paso a paso, inspeccionando variables). Antes de arrancar el cliente, asegúrate de que el servidor Spring Boot está en marcha en el puerto 8080.

14.1 Arrancar desde la terminal

En la terminal integrada, ejecuta el script de arranque. La aplicación quedará escuchando y podrás abrirla en el navegador.

Crear en: Terminal integrada de IntelliJ

Arrancar el cliente

```
1 npm run dev
```

Abre `http://localhost:3000/login` en el navegador, inicia sesión con un usuario válido del servidor y comprueba el listado, el alta, la edición y la baja.

14.2 Una configuración de ejecución para depurar

Crea la configuración en *Run* → *Edit Configurations* → *+* → *Node.js*. En «JavaScript file» indica `src/app.js`. Guarda y usa el botón de depuración (el del insecto): puedes poner puntos de interrupción (breakpoints) haciendo clic en el margen izquierdo del editor, y el IDE detendrá la ejecución allí para que inspecciones las variables.

Verificar las llamadas a la API

Si algo no funciona, revisa la consola de IntelliJ: los errores de Axios muestran el código de estado y el cuerpo de la respuesta del servidor. Un 401 indica token ausente o vencido; un 404, un endpoint mal escrito; un error de conexión, que el servidor Spring Boot no está en marcha.

15 El proyecto completo

Hemos resuelto cada pieza por separado; reunidas, forman la aplicación cliente completa. El siguiente mapa relaciona cada requisito con las piezas que lo resuelven, y la estructura muestra el conjunto de archivos.

Requisito	Piezas que lo resuelven
Inicio de sesión JWT	authServicio.js + rutas/auth.js + login.ejs + la sesión.
Envío del token	Interceptor de Axios (cabecera Authorization: Bearer).
Control de acceso	Middleware requiereSesion sobre las rutas privadas.
CRUD	estudianteServicio.js + rutas/estudiantes.js + lista.ejs y form.ejs.
Manejo de errores	Middleware de error en app.js (401 al login, 500 genérico).

Estructura del proyecto cliente

```

1 cliente-estudiantes/
2 |- package.json
3 |- .env # API_BASE_URL, SESSION_SECRET, PORT
4 |- .gitignore # incluye .env y node_modules
5 |- src/
6   |- app.js # servidor Express (middlewares y rutas)
7   |- rutas/
8     |- auth.js # login, logout
9     |- estudiantes.js # CRUD
10    |- guardia.js # requiereSesion
11   |- servicios/
12     |- apiCliente.js # instancia de Axios + interceptor
13     |- apiFetch.js # alternativa con fetch nativo
14     |- authServicio.js # iniciarSesion
15     |- estudianteServicio.js # listar, buscar, crear, actualizar, eliminar
16   |- vistas/
17     |- login.ejs lista.ejs form.ejs
18   |- publico/
19     |- estilos.css

```

Con esto, el cliente Node.js consume de extremo a extremo los servicios web de Spring Boot: inicia sesión y obtiene el token JWT, lo envía en cada petición protegida, y realiza el CRUD completo mostrando los datos en vistas. Los conceptos que dominas aquí —REST, JSON, JWT, middlewares, promesas— se aplican igual frente a cualquier API REST, sea cual sea la tecnología del servidor.

16 Glosario

Términos y siglas que aparecen en la guía, para consulta rápida. Los términos en inglés incluyen su significado en español.

Término	Significado
API	Interfaz de programación de aplicaciones; el contrato del servicio.
REST	Estilo de arquitectura para servicios web sobre HTTP.
Endpoint	Punto de acceso: una dirección concreta de la API.
HTTP	Protocolo de petición–respuesta de la web.
JSON	Notación de objetos de JavaScript; formato de intercambio de datos.
Payload	Carga útil: el cuerpo de datos de una petición o respuesta.
Header	Cabecera: par nombre–valor de control en la petición/respuesta.

Término	Significado
JWT	Token web JSON; credencial firmada de autenticación.
Bearer	Portador: esquema de la cabecera Authorization (Bearer <token>).
Runtime	Entorno de ejecución (Node.js ejecuta JavaScript fuera del navegador).
npm	Gestor de paquetes de Node; instala dependencias.
Dependencia	Librería externa que el proyecto usa.
Framework	Marco de trabajo que estructura la aplicación (Express).
Middleware	Software intermedio: función que procesa la petición en cadena.
Ruta (route)	Asociación de un verbo y una URL con el código que responde.
Motor de plantillas	Herramienta que combina HTML y datos para generar la página (EJS).
Promesa (promise)	Objeto que representa un resultado futuro de una operación asíncrona.
async/await	Sintaxis para escribir código asíncrono de forma secuencial.
Interceptor	Función que se ejecuta antes de cada petición de Axios.
Sesión (session)	Almacén por usuario en el servidor entre peticiones.
CORS	Intercambio de recursos de origen cruzado; norma de seguridad del navegador.
BFF	Backend para el frontend; backend propio que sirve al cliente web.
Serializar	Convertir un objeto a texto (JSON); deserializar es el paso inverso.
PRG	Post-Redirect-Get; redirigir tras modificar para evitar reenvíos.

17 Recursos

Para profundizar, estos son los recursos de referencia más fiables.

- Node.js (documentación oficial y API de `fetch`): nodejs.org/docs

- Express: **expressjs.com**
- EJS: **ejs.co**
- Axios: **axios-http.com**
- Estándar HTTP y JWT (IETF): **rfc-editor.org**
- OWASP (seguridad de aplicaciones web): **owasp.org**

Referencias

- [1] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Ph.d. dissertation, University of California, Irvine, 2000. URL <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [2] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. Request for Comments RFC 9110, Internet Engineering Task Force (IETF), June 2022. URL <https://www.rfc-editor.org/info/rfc9110>. STD 97.
- [3] Tim Bray. The JavaScript object notation (JSON) data interchange format. Request for Comments RFC 8259, Internet Engineering Task Force (IETF), December 2017. URL <https://www.rfc-editor.org/info/rfc8259>. STD 90.
- [4] Mario Casciaro and Luciano Mammino. *Node.js Design Patterns: Design and Implement Production-grade Node.js Applications Using Proven Patterns and Techniques*. Packt Publishing, Birmingham, UK, 3rd edition, 2020. ISBN 9781839214110.
- [5] WHATWG. Fetch — living standard. Web Hypertext Application Technology Working Group, 2024. URL <https://fetch.spec.whatwg.org/>.
- [6] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force (IETF), May 2015. URL <https://www.rfc-editor.org/info/rfc7519>.
- [7] Yaron Sheffer, Dick Hardt, and Michael B. Jones. JSON web token best current practices. Request for Comments RFC 8725, Internet Engineering Task Force (IETF), February 2020. URL <https://www.rfc-editor.org/info/rfc8725>. BCP 225.
- [8] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. Open Worldwide Application Security Project, 2021. URL <https://owasp.org/Top10/>.